

IQMS — Bug Fixes

This document is a focused changelog of the verified bug fixes and operational improvements currently present in the local codebase.

It is intentionally separate from `DEVELOPER_DOC.md` so developers can review bug history without scanning the full architecture and setup guide.

Scope

Verified against the current local implementation in:

- `Head-Detector/main.py`
 - `Head-Detector/utils/db_logger.py`
 - `Queue-Management-System-v2-main/Queue-Management-System-v2-main/queue_management_v2.py`
 - `Queue-Management-System-v2-main/Queue-Management-System-v2-main/prophet_predict.py`
 - `Queue-Management-System-v2-main/Queue-Management-System-v2-main/ensemble_predict.py`
 - `Queue-Management-System-v2-main/Queue-Management-System-v2-main/utils/db_logger.py`
 - `simulator/engine.py`
 - `simulator/entities.py`
 - `simulator/server.py`
 - `simulator/app.py`
 - `simulator/predict_viz.py`
 - `simulator/frontend/index.html`
-

Summary

The current local version includes fixes for:

- tracker update duplication in Head-Detector
- table/schema mismatch between pipelines
- excessive dwell update writes
- forecast training-data precedence issues
- mixed time granularity in forecasting
- hardcoded timezone shift in forecasting
- Grafana/Timescale incompatibilities caused by `TIMESTAMP`
- undercounting in v2 when no face was matched
- repeated model retraining overhead
- fixed wait-time estimation based only on predicted arrivals

The most recent improvements also add:

- `queue_state_snapshots` hypertable
 - `service_events` hypertable placeholder for future exit-line logic
 - dynamic wait estimation based on snapshot state
 - `wait_15m` and `wait_30m` forecast horizon outputs
-

Verified Fixes

1. Tracker called multiple times per frame in Head-Detector

Problem

The Norfair detection build and `tracker.update()` flow used to run inside the per-box loop, so one frame with multiple detections could trigger multiple tracker updates and duplicate downstream processing.

Fix

Tracker input is now built once after all ROI-valid boxes are collected, and `tracker.update()` is called once per frame.

Current result

- cleaner tracking state
 - no duplicate insert/update behavior from multi-call tracker execution
-

2. Table mismatch between pipelines

Problem

The Head-Detector pipeline previously used a different table/method pattern from the forecasting and v2 pipeline, which prevented all components from reading the same event stream.

Fix

Both pipelines now write to `entrance_events`, and both DBLogger classes use:

```
insert_entrance(track_id, gender, age, confidence, camera_id)
```

Current result

- both pipelines feed the same forecasting and Grafana data source
-

3. Dwell updates firing too often

Problem

Using `int(track_dur) % 2 == 0` caused multiple UPDATE calls within the same 2-second window at normal frame rates.

Fix

Both pipelines now track the last dwell write time using `last_dwell_times` and only update when at least 2.0 real seconds have elapsed.

Current result

- lower DB write pressure
 - more stable dwell values
-

4. Forecast scripts preferring simulated data over real data

Problem

When simulated rows were concatenated before real rows, `drop_duplicates()` kept the simulated version and discarded real observations for overlapping timestamps.

Fix

Both forecast scripts now place real rows first before deduplication.

Current result

- real measurements win whenever timestamps overlap
-

5. Mixed forecast granularity

Problem

Real data and simulated history were generated at different time resolutions, producing an irregular training series.

Fix

Both forecast scripts now use:

```
time_bucket('5 minutes', timestamp)
```

for real DB aggregation, matching the 5-minute simulation grid.

Current result

- regular forecast training frequency
 - more consistent Prophet/XGBoost/LSTM inputs
-

6. Hardcoded timezone offset in forecasting

Problem

Forecast generation previously relied on a fixed timezone shift hack, which broke on systems outside the assumed timezone.

Fix

Future timestamps are now anchored directly to local `pd.Timestamp.now()`.

Current result

- correct local forecast horizon alignment
-

7. Grafana and TimescaleDB schema incompatibility

Problem

Legacy tables using `TIMESTAMP WITHOUT TIME ZONE` caused Grafana time-field detection issues and conflicted with Timescale hypertable expectations.

Fix

Both DB logger implementations and the forecast writer now:

- use `TIMESTAMPTZ`
- create Timescale hypertables automatically

- run migration blocks to convert legacy timestamp columns
- avoid incompatible SERIAL PRIMARY KEY usage on hypertables

Current result

- Grafana can detect time fields correctly
 - TimescaleDB hypertable creation works cleanly
-

8. v2 silently dropping body-only detections

Problem

queue_management_v2.py previously only sent detections to Norfair when a face was matched to the body. Occluded faces were therefore missed in tracking and DB inserts.

Fix

The face-required guard was removed. Every ROI-valid body detection is now passed into the tracker, while unmatched detections fall back to:

- gender='unknown'
- age=None

Current result

- better queue counts under occlusion
 - improved compatibility with forecasting and live metrics
-

9. Repeated retraining overhead in ensemble forecasting

Problem

The ensemble script retrained LSTM and XGBoost from scratch on every run, increasing runtime and making frequent scheduling expensive.

Fix

The script now persists and reloads:

- models/lstm_queue.keras
- models/lstm_scaler.pkl
- models/xgb_queue.json

using a freshness threshold controlled by MODEL_MAX_AGE_HOURS.

Current result

- much faster repeated forecast runs
 - lower operational overhead for scheduled prediction jobs
-

10. LSTM scaling distorted by simulated history

Problem

Fitting the scaler on combined simulated + real data could compress the real observed range.

Fix

The LSTM scaler is now fit on real data when available, and only falls back to combined fitting when no real data exists yet.

Current result

- better preservation of real traffic scale
-

11. Dynamic wait-time estimation added

Problem

Wait time was previously approximated with a fixed formula:

$$(\text{entries} * 3) / 2$$

This ignored actual queue backlog, observed dwell, and lane count.

Fix

The current `ensemble_predict.py` now:

- reads the latest row from `queue_state_snapshots`
- seeds `current_queue`, `avg_dwell_sec`, and `active_lanes`
- derives `service_per_bucket`
- rolls queue backlog forward over each forecast bucket
- computes `est_wait_minutes`, `wait_15m`, and `wait_30m`

Current result

- wait estimation reflects live queue state
 - queue predictions now expose short-horizon wait outputs
-

New Supporting Tables

[queue_state_snapshots](#)

Added to both DB logger implementations.

Purpose:

- periodic queue-state logging
- dynamic wait estimation seeding
- live Grafana queue-state panels

Stored fields include:

- `camera_id`
- `queue_count`
- `avg_dwell_sec`
- `max_dwell_sec`
- `active_lanes`

[service_events](#)

Added to both DB logger implementations.

Purpose:

- placeholder for future exit-line / service completion logging

Current state:

- schema and logger method exist
 - not yet populated automatically by the pipelines
-

Simulator — Fixes and Improvements

These apply to `simulator/` and were introduced after the initial simulator implementation.

S1. `st.components.v1.html` deprecation warning

Problem

`app.py` embedded the canvas frontend using `streamlit.components.v1.html`, which Streamlit deprecated (removal after 2026-06-01) and warned on every page rerun.

Fix

Removed `_load_frontend_html()` and `_render_frontend()`. The frontend is now served as a static file via a Python `http.server` subprocess on port 8081. The Streamlit tab uses `st.iframe("http://localhost:8081/index.html")` to embed it.

S2. `WebSocket EOFError` spam on port check

Problem

`is_port_in_use()` originally used `socket.connect_ex()`. On every Streamlit rerun this opened a TCP connection to the running WebSocket server, which logged stream ends after 0 bytes EOFErrors.

Fix

Port check now uses `socket.connect_ex` with a short timeout (`settimeout(0.2)`) across both IPv4 and IPv6, preventing stalled checks without triggering WebSocket errors.

S3. `WebSocket handler` crash when `Streamlit client` disconnected

Problem

The start action handler used `await websocket.send(...)` to confirm the simulation started. The Streamlit command client closes immediately after sending; the `send()` call on the already-closed socket raised a `ConnectionClosedError`.

Fix

All server-to-client broadcasts replaced with `websockets.broadcast(connected, msg)`, which silently skips closed connections.

S4. `Prophet` predicting queue counts at closed hours

Problem

Forecast training data had no rows for closed hours (e.g. 22:00–08:00). Prophet interpolated across the gap and predicted non-zero queue counts at night.

Fix

`predict_viz.py` now:

- Fills closed hours with explicit zero rows during training (`_add_closed_zeros`)
 - Only generates future forecast timestamps during open hours (`_future_open_timestamps`)
 - Uses `SHOP_OPEN = 8`, `SHOP_CLOSE = 21` constants shared across training and forecast steps.
-

S5. Simulation restart on sidebar slider change

Problem

Slider changes triggered a full Streamlit rerun, which remounted the `st.iframe` and reset the canvas JavaScript state, visually restarting the simulation.

Fix

The simulator tab is wrapped in `@st.fragment` so only internal widget interactions (Start / Stop buttons) cause the `iframe` to remount.

S6. Queue History chart empty in prediction tab

Problem

`_load_snapshots()` filtered by source (REAL/SIM/ALL). When source was REAL but only simulator snapshots existed, the query returned nothing.

Fix

Snapshot history is now loaded regardless of the source filter (snapshots are always relevant to queue history display).

S7. Entrance dwell stage missing from simulation

Problem

Customers went directly from ARRIVAL to shop floor with no modelling of time spent at the store entrance (collecting a trolley, picking bakery items, group coordination).

Fix

Added `ENTRANCE_DONE` event and `people_at_entrance` state to the engine.

Entrance dwell is computed per customer:

- base 5–15s pass-through
- +30–90s if `has_caddy`
- +20–90s if `picks_bakery`
- +10–25s if `is_group`

`picks_bakery`, `entrance_eta`, `entrance_done_time`, and `entrance_dwell_sec` fields added to `Person`.

Customers at entrance are broadcast as a separate `entrance[]` array in the WebSocket update and rendered in the entrance panel of the canvas with zone clustering (bakery top / caddy bottom / passing-through centre).

April 2026 — Data Quality & DB Pollution Fixes

12. Tracker re-detection spam inflating entrance counts

Problem

When a person was briefly occluded (1–2 frames), Norfair dropped and re-created the track. Each new track triggered an immediate `insert_entrance`, producing hundreds of spurious entries per 3-minute bucket (observed peak: 644 entries in one bucket vs a normal 10–30).

Root cause

`insert_entrance` was called on the first frame a new `track_id` appeared — before any confirmation that the detection was stable.

Fix — `queue_management_v2.py`

Track lifecycle completely redesigned:

- Accumulate all per-frame readings in `track_data[track_id]` while the track is alive
- **Insert once at track death** (when the `track_id` disappears from the tracker)
- Timestamp written is the **track start time** (`entry_dt`), not the insert time — preserving correct arrival bucket assignment
- Silently discard tracks with total dwell < `min_elapsed_time` (`config2.yml`, default 1s)

Fix — `db_logger.py`

`insert_entrance` now accepts optional `dwell_seconds`, `entry_time`, and `has_bag` parameters:

```
def insert_entrance(self, track_id, gender, age, confidence, camera_id,
                    dwell_seconds=None, entry_time=None, has_bag=False)
```

When `entry_time` is provided it is written as the row timestamp instead of `NOW()`.

Current result

- no more NULL `dwell_seconds`
 - no more duplicate entries from brief occlusions
 - arrival timestamps reflect actual entry time not exit time
-

13. Gender and age using only first-frame reading

Problem

Gender and age were captured from the first confirmed frame. If that frame had a partial view (turned head, low confidence), the stored value was unreliable for the entire track.

Fix

All face readings during the track lifetime are now accumulated:

- **Gender:** confidence-weighted vote — each frame adds `face_confidence` to the detected gender's running total; the winner at death is stored

- **Age:** confidence-weighted average across all frames where a valid age was detected

```
track_data[track_id] = {
    "gender_votes": {"male": 0.0, "female": 0.0, "unknown": 0.0},
    "age_sum": 0.0,
    "age_weight": 0.0,
    "best_conf": 0.0,
    "entry_dt": datetime.now(),
}
```

Current result

- gender/age represent the best consensus across the full track lifetime
 - a person seen 10× as male (conf 0.9 each) wins over 2× ambiguous female readings
-

14. Test-mode camera_id written as real data

Problem

Both queue_management_v2.py and Head-Detector/main.py set camID = 'live_test_video' when running against the test video file. This ID does not start with SIM_, so all test-run entries were counted as REAL data in training queries.

Fix

Changed to camID = 'SIM_live_test_video' in both files so the NOT LIKE 'SIM_%' filter correctly excludes test runs.

15. Historical DB pollution from restart duplicates

Problem

On app restart, Norfair resets track_ids from 0. People in frame at restart were re-inserted with new IDs. Combined with the pre-fix immediate insert, the same person could accumulate many entries on the same day.

Cleanup applied (2026-04-12)

```
DELETE FROM entrance_events
WHERE camera_id NOT LIKE 'SIM_%'
AND id NOT IN (
    SELECT MIN(id)
    FROM entrance_events
    WHERE camera_id NOT LIKE 'SIM_%'
    GROUP BY track_id, camera_id, DATE(timestamp)
);
-- Deleted 8610 rows

DELETE FROM entrance_events
WHERE camera_id NOT LIKE 'SIM_%'
AND dwell_seconds IS NULL;
-- Deleted 10791 rows (brief re-detections never updated)
```

Prevention

The track-at-death architecture (fix 12) eliminates the root cause — only confirmed tracks with a valid dwell are ever written.

16. Camera rename — historical rows updated

2026-04-12: both DB tables updated to match new config names:

Old name	New name
Bosch_Camera_169	Bosch_Camera_Entrance
Bourgogne_Sortie_Gauche	Bosch_Camera_Exit

Applied to entrance_events (9936 rows) and queue_state_snapshots (7174 rows).

17. Forecast bucket reduced from 5 min to 3 min

Change

All `time_bucket('5 minutes', ...)` SQL calls and matching pandas `freq="5min" / .floor("5min")` calls updated to 3 minutes / 3min across:

- `predict_viz.py` — training query, live actuals query, in-sample grid, components grid
- `app.py` — diagnostics spike query
- `_add_closed_zeros()` — closed-hour padding step interval

Reason

3-minute granularity gives finer peak resolution while remaining stable enough for Prophet with the current data volume (~10–50 entries per bucket at normal traffic).

18. Bucket enriched with gender, age and dwell aggregates

Change

Both `_load_arrivals()` and `load_today_actuals()` in `predict_viz.py` now return per-bucket aggregates alongside entry count:

```
COUNT(*) FILTER (WHERE gender = 'male')           AS male_count,
COUNT(*) FILTER (WHERE gender = 'female')        AS female_count,
ROUND(AVG(age_estimate)::numeric, 1)              AS avg_age,
ROUND(AVG(dwell_seconds) FILTER (WHERE dwell_seconds > 0)::numeric, 1) AS avg_dwell_sec
```

Note: `male_count`, `female_count`, and `avg_age` are only populated for `Bosch_Camera_Entrance` (Pipeline B with uniface). `Bosch_Camera_Exit` (Head-Detector) stores `gender='unknown'` and `age=NULL`.

19. Bag detection added to Pipeline B (COCO classes 24 + 26)

Change

`queue_management_v2.py` now detects handbags and backpacks using the YOLOv9 ONNX model alongside person detection.

Implementation

```
BAG_CLASSES = {24, 26}  # COCO: 24 = backpack, 26 = handbag (28 = suitcase, excluded)
```

On each frame, bag bounding boxes are collected separately from person boxes. A person is associated with a bag if the bag centre falls inside a 30%-expanded person bounding box (`_person_has_bag()`). The `has_bag` flag is latched: once `True` for a given `track_id` it never resets to `False` — ensuring a bag carried into one frame is not lost if the bag is briefly occluded.

At track death, the resolved `has_bag` value is passed to `db_logger.insert_entrance()`.

Schema change — db_logger.py (Head-Detector)

Added `has_bag` `BOOLEAN` `DEFAULT FALSE` to `entrance_events`. An idempotent migration block adds the column to existing tables automatically.

Updated `insert_entrance` signature:

```
def insert_entrance(self, track_id, gender, age, confidence, camera_id,
                    dwell_seconds=None, entry_time=None, has_bag=False)
```

Note: `has_bag` is now populated by both the simulator (`simulator/db.py`, always was) and Pipeline B (`queue_management_v2.py`). Pipeline A (Head-Detector) does not perform bag detection and always stores `has_bag=False`.

Current Limitations Still Present

- `service_events` exists but is not yet populated by a true exit-line workflow in the live production pipelines
 - `track_id` remains session-local and resets on app restart (harmless since insert-at-death deduplicates naturally)
 - `prophet_predict.py` remains a console-only forecast helper and does not write predictions to `queue_predictions`
 - the simulator writes into shared production tables using `SIM_*` camera IDs rather than dedicated simulator-only tables
 - `gender/age` enrichment in buckets only meaningful for Pipeline B camera; Pipeline A always stores `unknown/NULL`
-

Recommended Next Step

If bug tracking will continue to evolve, keep:

- `DEVELOPER_DOC.md` for system documentation
- `BUG_FIXES.md` for verified fix history

This keeps architecture, operations, and change history separate and easier to maintain.